

it is essential for us to have a good understanding of the popular jargon in these fields. That is why I believe our discussions on AI, machine learning, data science, neural networks, and related topics are well justified.

Recall that in the last few articles in this series, we thoroughly covered neural networks. We talked about using TensorFlow and Keras to build and test models based on neural networks. However, it's important to note that our approach to neural networks was largely practical in nature. Remember that in the previous two articles in this series, we created and examined a neural network model capable of classifying images of handwritten digits. Now, I believe it is the right time to delve into the various types of neural networks that can be employed in such models.

The origin of neural networks can be traced back to the early 1940s, prior to the development of digital computers itself. In 1943, Warren McCulloch and Walter Pitts introduced perceptron (also called McCulloch-Pitts neuron), a mathematical representation of a simple neuron capable of processing and transmitting information like the neurons in the human brain. Despite the groundbreaking proposal, neural networks didn't immediately become the dominant technology in the field of AI. Instead, other techniques such as support vector machines and linear regression were favoured in the development of AI and machine learning applications. It wasn't until the 1990s that neural networks saw a resurgence in popularity. In the next section, we will explore some of the commonly used neural networks and list the corresponding Python libraries for implementing them.

First, let us discuss feedforward neural networks (FNN). FNNs are the simplest of all the neural networks. In an FNN, information flows only in one direction — from the input layer to the output layer. These networks are used for a wide range of tasks,

including image classification and regression analysis. A single-layer perceptron is one of the simplest FNNs, consisting of just an input layer and an output layer. They are often used for supervised learning of binary classifiers. There are also multilayer perceptrons which are far more powerful than single-layer perceptrons. Unlike single-layer perceptrons, multilayer perceptrons have one or more hidden layers in addition to an input layer and an output layer. Multilayer perceptrons are often used for regression analysis. TensorFlow/Keras and PyTorch can be used to implement FNNs in Python. Consider the Python script named *mlp.py* shown below which uses a library called *torchvision* — part of the PyTorch framework. If this library is not part of the version of PyTorch installed in your system, use Anaconda Navigator to install it. The Python script *mlp.py* generates a multi-layer perceptron (MLP). Figure 1 shows the output of the Python script named *mlp.py*.

```
import torch
import torchvision.ops as ops

mlp = ops.MLP(in_channels=5, hidden_channels=[5, 5])
inp = torch.randn(1, 5)
print(inp)
output = mlp(inp)
```

```
# python mlp.py
tensor([[[0.0796, 1.3288, 0.5934, 1.1947, 0.8144]])
tensor([[-0.1073, -0.1165, -0.0880, 0.8064, 0.2044]],
        grad_fn=<AddmmBackward0>)
```

Figure 1: The working of a multilayer perceptron

```
# python cnn.py
tensor([[[[ 0.2679]],

          [[ 0.0545]],

          [[ 0.9483]],

          [[-1.4280]]]])
tensor([[[[-0.6475]]], grad_fn=<SqueezeBackward1>)
```

Figure 2: The working of a CNN

```
print(output)
```

Now, let us discuss convolutional neural networks (CNN). A CNN is a type of FNN. CNNs are specifically designed to process image data. They are often used for image classification, object detection, image segmentation, etc. TensorFlow/Keras and PyTorch can be used to implement CNNs in Python. CNNs are made of a number of convolutional layers. Recall that our model for classifying images of handwritten digits used convolutional layers. Consider the Python script named *cnn.py* shown below. This Python script generates a 2D convolution layer using PyTorch. Figure 2 shows the output of the Python script named *cnn.py*.

```
import torch
import torch.nn as nn
model = nn.Conv2d(4, 1, 1)
inp = torch.randn(4, 1, 1)
print(inp)
output = model(inp)
print(output)
```

Now, let's discuss recurrent neural networks (RNN) and long short-term memory networks (LSTM). RNNs have loops that enable information to be passed from one step to the next. LSTM, on the other hand, is a more sophisticated type of RNN